

PART VI — THE SELF-LEARNING LAYER

Chapter 35

Interactive Session: CLI Commands for Learning

“The whole self-learning layer is only as usable as the three words that trigger it.”

Chapter 35 — Interactive Session: CLI Commands for Learning

Chapters 29 through 34 built the machinery — feedback capture, failure memory, distillation, hybrid retrieval, merge deduplication. None of it is directly reachable by a user typing a question. This chapter covers the thin, deliberately small command layer that actually exposes it: three keywords — ``bad``, ``stats``, ``learn`` — recognized inline in the same input stream as ordinary questions, dispatched to the functions Chapters 29 through 32 already built.

This chapter also looks at a design this project did not ship — a richer slash-command syntax (``/save``, ``/correct``, ``/forget``) — not as a missing feature, but as a genuine alternative worth weighing against the three-keyword design that was actually chosen, and asks what a CLI needs to tell a user about what just happened without becoming as verbose as the debug log sitting right below it.

It is worth naming upfront why a chapter on three short keywords deserves its own place in Part VI at all, rather than being folded as a footnote into the chapters that built the underlying mechanisms. Every mechanism in Chapters 29 through 34 is reachable from code — a Python test harness, a batch script, a direct function call — with no CLI involved whatsoever. But a real user sitting at a terminal has exactly one channel into this system: the text they type. If that channel cannot cleanly express "this was wrong," "show me the current learning state," or "learn from what just happened now," then the self-learning layer's actual usability is bottlenecked by an interface problem no amount of backend sophistication fixes.

35.1 The `bad` command

``user_input_node()`` is the single dispatch point every line of user input passes through: ``raw.lower()`` is checked against ``{"exit", "quit"}``, ``"stats"``, ``"learn"``, and ``"bad"`` before anything is treated as an ordinary query. Recognizing ``bad`` sets ``state["command"] = "bad"`` and routes to ``cmd_bad()`` — but ``bad`` alone carries no information about *what* was wrong, which is why it is inherently a two-step interaction: the keyword triggers a follow-up prompt asking for the actual feedback text, and that second input is what ``cmd_bad()`` actually forwards to ``mark_last_bad()``.

Figure 35.2 traces this two-step shape end to end, from the bare keyword through to the gated persistence call, because the gap between the two steps is precisely where Chapter 31.2's ``MIN_FEEDBACK_LEN`` threshold gets applied — a detail easy to miss if ``bad`` is thought of as one atomic action rather than the two genuinely separate inputs it actually is.

The `bad` command's two-step interactive prompt

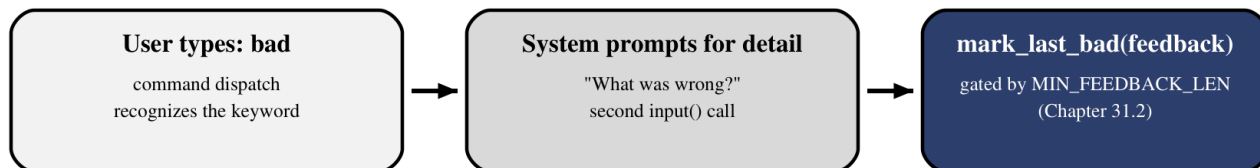


Figure 35.2 — The `bad` command's two-step shape mirrors the reality that a bare thumbs-down and a thumbs-down with an explanation trigger meaningfully different persistence (Chapter 31.2).

This two-step shape is not incidental friction — it is the exact seam Chapter 31.2's ``MIN_FEEDBACK_LEN`` gate depends on. A single-step ``bad <reason>`` command would work too, but the two-step version makes the *asking* for detail an explicit, visible part of the interaction, nudging a user who might otherwise type a bare ``bad`` toward providing the richer signal Chapter 31's entire mechanism is built to use.

There is a second, quieter reason the two-step design earns its keep: it separates *noticing* a problem from *articulating* it, which are genuinely different cognitive tasks happening at different moments. A

user reading a bad answer reacts first — the ``bad`` keyword captures that reaction immediately, before the moment passes and before ``mark_last_bad()`` needs anything more than the fact that the last interaction should be flagged. The follow-up prompt then gives that same user a second, separate moment to explain what was wrong, without requiring them to have composed that explanation before they had even finished reading the flawed answer. A single-step command would force both to happen at once, under time pressure, likely producing shorter and less useful feedback text as a result.

35.2 The stats command

``cmd_stats()`` is read-only and answers exactly three questions a user or operator might have about the system's learning state: how many interactions have been logged in total, how many were good enough to potentially feed distillation, and how many QA pairs currently live in ``learned_qa``.

```
def cmd_stats(state: GraphState) -> dict:
    total = feedback_store.count()
    good = feedback_store.count_good()
    learned = learned_collection.count()
    print(f"\n  Total interactions logged : {total}")
    print(f"    Successful (OK quality)      : {good}")
    print(f"    Learned QA pairs in store : {learned}")
    if get_switches(state)["ENABLE_AUTO_DISTILLATION"]:
        next_n = LEARN_EVERY_N - (good % LEARN_EVERY_N) if good > 0 else
LEARN_EVERY_N
        print(f"      Next distillation at          : {next_n} more good
interaction(s)\n")
    else:
        print("    Automatic distillation      : disabled\n")
    return {}
```

The "Next distillation at" line is a small but genuinely useful piece of derived information — it is not stored anywhere, it is computed on the fly from ``count_good()`` and ``LEARN_EVERY_N`` (Chapter 32.7's trigger constant), turning an abstract modulo condition into a concrete, human-readable countdown. This is the identical spirit as Chapter 27's state-engineering discipline, applied to a human-facing surface instead of a model-facing one: derive what is needed at the moment it is needed rather than maintaining a separately-tracked "turns until next distillation" counter that could drift from the real count.

Notice also what ``cmd_stats()`` deliberately does not report: it shows no breakdown of ``INSUFFICIENT`` or ``USER_THUMBSDOWN`` counts, no error rates, no latency figures. This is a scope decision, not an oversight — ``stats`` answers the specific question "is the self-learning layer working and how close is it to its next action," not the broader question "is this deployment healthy," which belongs to the observability stack Part IV's dry-run tracing already covers. Conflating the two would turn a three-line status check into something closer to a dashboard, at the cost of the quick, scannable answer ``stats`` is actually designed to give.

The conditional branch on ``ENABLE_AUTO_DISTILLATION`` (Chapter 36B.2) is worth reading closely too: when automatic distillation is switched off, showing a "next distillation at N more interactions" countdown would be actively misleading, since no such automatic event will ever fire regardless of how high the good-interaction count climbs. Reporting "Automatic distillation: disabled" instead is a small but real instance of a command telling the truth about what the system will actually do, rather than presenting a projection that quietly assumes a configuration the deployment may not have.

35.3 The learn command

`cmd_learn()` bypasses `should_learn()`'s automatic trigger entirely and calls `run_distillation()` directly, on demand. This is the manual strategy from Chapter 32.7's trigger table, and its value is independent of whether automatic distillation is even enabled — an operator who just finished a batch of known-good interactions (`run_batch.py`'s Batch 12 scripts exactly this: ask a question, force `learn`, then verify the learned count grew) does not want to wait for the count to naturally cross a multiple of `LEARN_EVERY_N`.

```
def cmd_learn(state: GraphState, config=None) -> dict:
    print("Forcing distillation now...")
    added = self_learner.run_distillation(config=config,
    switches=get_switches(state))
    print(f"Added {added} QA pair(s) to learned_qa.\n")
    return {}
```

Reporting the exact count added — not merely "distillation complete" — closes the loop a user just opened by typing the command: they asked for something to happen, and the response confirms specifically what happened, including the honest zero-pairs-added case Chapter 32.6 already covered as a legitimate, non-error outcome.

`run_batch.py`'s Batch 12 script demonstrates why this immediate confirmation matters operationally, not just conversationally: it asks a baseline question, checks `stats`, forces `learn`, checks `stats` again, then re-asks a related question to see whether the newly learned entry gets retrieved. Every step of that sequence depends on the *previous* step's output being trustworthy and immediately visible — a `learn` command that silently succeeded with no count reported would break the script's ability to verify anything at all, since there would be no way to confirm the forced distillation actually did what it claimed before moving on to test its effect.

35.4 The /save, /correct, /forget pattern

A richer alternative design is worth considering explicitly, even though this project did not build it: slash-prefixed commands, `/save` to force distillation of the last answer regardless of the automatic trigger, `/correct <text>` to combine a thumbsdown with an immediate replacement answer in one step, and `/forget <query>` to explicitly remove a specific entry from `learned_qa` rather than waiting for it to age out or be superseded.

Design	Command surface	Trade-off
Plain keywords (shipped)	bad, stats, learn, exit	Minimal, no parsing ambiguity with real questions
Slash commands (alternative)	/save, /correct, /forget, /stats	More expressive; needs a real parser and collides less by design

The plain-keyword design this project shipped has one advantage the slash-command alternative gives up for free: `bad`, `stats`, and `learn` are recognized by exact string match against `raw.lower()`, with no parsing beyond a dictionary lookup, and no risk of a legitimate question accidentally starting with a reserved prefix. A slash-command design trades that simplicity for expressiveness — `/correct <replacement text>` genuinely cannot be expressed in the three-keyword vocabulary at all, since there is no way to attach a payload to `bad` beyond the two-step prompt Section 35.1 already uses for feedback text specifically.

`/forget` is the most consequential of the three hypothetical commands, because it is the one closest to Chapter 28.5.1's argument that "a system that can add experience but cannot locate and remove it is accumulating liability, not learning responsibly." This project's shipped design has no user-facing removal path for a specific `learned_qa` entry at all — an operator wanting to remove one would need to interact with the ChromaDB collection directly, outside any CLI command this chapter covers. `/forget` names a real, currently-unaddressed gap, not a nice-to-have.

`/save` and `/correct` are lower-stakes but still genuinely useful gaps in the shipped vocabulary. `/save` would let a user force distillation of one specific answer they found valuable, independent of whether that interaction happened to land on a `LEARN_EVERY_N` boundary or whether an operator remembers to run the blanket `learn` command afterward — a more surgical version of Section 35.3's forced distillation, scoped to one interaction instead of the whole eligible batch. `/correct <replacement>` goes further still, combining a thumbsdown with an immediate, user-supplied better answer in a single step, rather than requiring the system to blindly re-retrieve and hope the next attempt lands closer to what the user actually wanted — effectively letting the user write the `learned_qa` entry directly instead of leaving it entirely to distillation's own judgment.

None of these three hypothetical commands would be difficult to build on top of the machinery Chapters 29 through 34 already provide — `/save` is a thin wrapper around a single-interaction call to `_generate_qa_pairs()` and `_upsert_pairs()`, `/forget` is a `collection.delete(ids=[...])` call away, and `/correct` mostly needs a way to accept multi-line input for the replacement text. The reason to walk through them here is not that they are hard, but that naming a command surface explicitly — even one that was never built — is itself a useful exercise: it makes clear which capabilities this project's self-learning layer structurally supports today and which ones would require new user-facing entry points before they could ever be used, no matter how ready the underlying storage and retrieval logic already is.

35.5 User-friendly logging and progress messages

Every command in this chapter prints directly to the user-facing terminal — `print()`, not `logger.debug()` — a deliberate separation this project maintains throughout: the debug log (Chapter 22) is the detailed, developer-facing trace of exactly what happened at every stage; the terminal output a command produces is the short, human-facing summary of what the user actually needs to know right now.

Figure 35.1 revisits this chapter's opening dispatch diagram from the logging angle specifically: every one of its four destinations owns its own small, self-contained burst of `print()` output, and none of them depends on the caller enabling any particular log level to be seen. That independence from logging configuration is itself a design requirement, not a coincidence — a CLI command whose confirmation only appears at DEBUG level has effectively failed silently for any user running at the default INFO level.

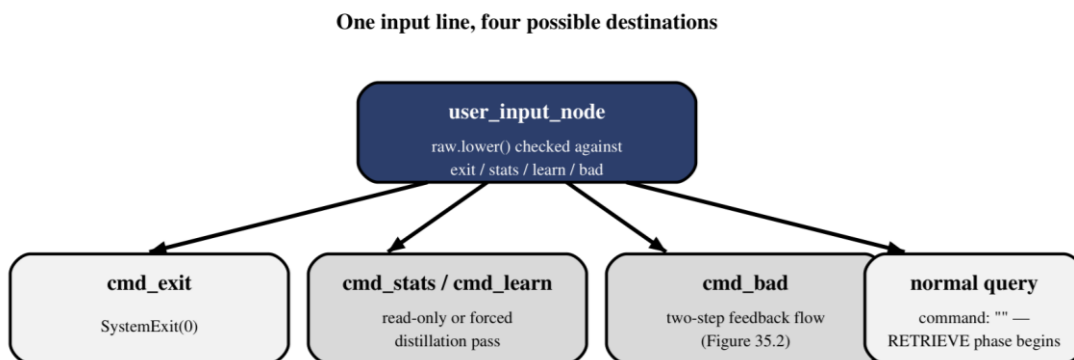


Figure 35.1 — Every line of input passes through one dispatch point before becoming either a special command or an ordinary query.

Common pitfall — Routing operator commands through the debug log

A ``stats`` or ``learn`` command whose only output is a DEBUG-level log line is invisible to a user running the CLI with default logging levels (Chapter 23's ``console_level=INFO`` default). Commands a user directly invokes need a ``print()``-level response guaranteed to reach the terminal, independent of whatever log verbosity happens to be configured.

This distinction generalizes past this specific project: any interactive command a user directly triggers should confirm what happened in plain, immediate terms — count added, feedback persisted or rejected, distillation forced — while the detailed **why** and **how** stays in the log stream for whoever needs to debug a problem later. Chapter 22's dry-run trace and this chapter's command output are not competing for the same audience, and conflating them serves neither well.

Chapter 36 asks the question this entire self-learning layer has been building toward and this chapter's ``stats`` command only partially answers: not just how many interactions were logged or how many pairs were distilled, but whether any of it actually made the system's answers measurably better.

That gap is worth stating plainly as this chapter closes. ``stats`` reports activity — counts of interactions, counts of learned pairs — not outcomes. A deployment could show a healthy, steadily growing ``learned_qa`` count while its actual answer quality stayed flat or even regressed, and nothing in this chapter's command surface would surface that discrepancy. Activity metrics are necessary for operating the system day to day, but they are not evidence of the thing the system is meant to achieve, which is precisely the distinction Chapter 36 exists to draw out.